

Overview of File Handling

File handling is the process of using Python to create, read, update, and delete files on your system. It allows your programs to store data permanently rather than losing it when the program stops running.

1. Opening a File (The Modes)

To work with a file, you use the `open()` function. You must specify a mode to tell Python what you intend to do:

- **Read Mode ('r'):** This is the default. It opens the file for reading. If the file does not exist, it will throw an error.
 - **Write Mode ('w'):** This opens the file for writing. If the file already exists, it overwrites everything. If it doesn't exist, it creates a new one.
 - **Append Mode ('a'):** This is safer than write mode. It adds new data to the end of the file without deleting what is already there.
 - **Create Mode ('x'):** This creates a file but returns an error if the file already exists.
-

2. Essential Operations

Reading Data

- `read()`: Reads the entire content of the file at once.
- `readline()`: Reads only the first line of the file.
- `readlines()`: Reads the whole file and puts each line into a list.

Writing Data

- `write()`: Inserts a string into the file (remember that in 'w' mode, this replaces everything).
 - `writelines()`: Allows you to insert a list of strings into a file at once.
-

3. The Best Way: Using the 'with' Statement

The most professional way to handle files is using the `with` keyword. This is called a Context Manager.

The main benefit is that it automatically closes the file for you as soon as the code block is finished. This prevents memory leaks and file corruption.

```
Example: > with open("notes.txt", "r") as file: data  
= file.read()
```

4. Working with Different Data Formats

CSV Files CSV (Comma Separated Values) files are used for data that looks like a spreadsheet. Python uses the `csv` module to handle these. You can read rows as lists or dictionaries, making it easy to process large amounts of data.

JSON Files JSON is the standard format for web data and APIs. Python uses the `json` module for two main tasks:

- **Serialization (dump):** Converting a Python dictionary into a JSON file.
 - **Deserialization (load):** Taking a JSON file and turning it back into a Python dictionary.
-

5. Safety and Exceptions

When handling files, things often go wrong (e.g., a file is missing or a folder is restricted). It is best practice to wrap your file code in a try-except block to catch `FileNotFoundError`.

-